FORGEGUARD

Autonomous AI Code Vulnerability Scanner, STRIDE Threat Modeler & Patch Engine

Hackathon: ForgeHacks 2026	**Track:** AI + Cybersecurity
Team / Project: SwarmForge / ForgeGuard	**Execution:** Tri-Agent Autonomous Engine
Status: Production Ready & Tested	**Target:** Sub-10ms SAST & AST Verification

1. The Enterprise Security Dilemma

Modern software delivery moves faster than security reviews can keep up, leading to critical vulnerabilities in production.

■ *Alert Fatigue & Lag*

Traditional SAST tools flood engineers with unverified false positives. Devs spend hours triaging alerts rather than writing secure code.

■ *Manual Threat Modeling*

STRIDE threat modeling is typically conducted on whiteboards months before launch and becomes immediately obsolete as code changes.

■ *High Remediation Cost*

Fixing a bug in production costs 30x more than during development. Security patches often break syntax or introduce regression bugs.

■ *Vulnerability Blindspots*

SQL injections, shell injection, insecure deserialization, and hardcoded API tokens remain rampant in rapid release cycles.

2. The ForgeGuard Solution

An end-to-end autonomous cybersecurity platform that combines high-precision AST SAST scanning, dynamic STRIDE threat modeling, and self-healing AST-verified code remediation.

1. High-Speed AST Scanner	Zero-dependency Python AST traversal + Taint flow tracking detects OWASP Top 10 vulnerabilities in milliseconds with precise line and column references.
2. STRIDE Threat Engine	Automatically maps all entry points and attack vectors into the 6 STRIDE categories with clear blast-radius calculations and architectural mitigations.
3. Autonomous Patcher	Synthesizes clean unified diffs, verifies AST syntax validity before touching disk, and maintains 100% snapshot rollback safety.
4. Live Interactive UI	Cyberpunk dark-mode dashboard with real-time CVSS gauges, interactive diff viewer, and 1-click remediation triggers.

3. Tri-Agent Collaborative Architecture

Three specialized autonomous AI agents collaborate in real-time to analyze, model, and patch codebases.

Agent 1: Security Auditor	*Agent 2: Threat Modeler*	*Agent 3: Remediation Engineer*
• Python AST Visitor & Lexer	• Full STRIDE Framework Analysis	• Autonomous Unified Diff Engine
• Taint-Flow Variable Tracking	• Asset Blast Radius Calculation	• Pre-Commit AST Syntax Checker
• CVSS 3.1 Base Metric Scorer	• Threat Vector Surface Mapping	• Atomic File Modification
• CWE & OWASP Top 10 Mapping	• Architectural Boundary Defense	• Instant Snapshot Rollback Safety
• Zero False-Positive Suppression	• Executive Risk Level Rating	• Zero Broken Builds Guarantee

4. STRIDE Threat Modeling in Action

Automating enterprise security posture assessment across all 6 core STRIDE pillars in real-time:

Category	*Target Threat*	*Mitigation Strategy*
Spoofing (S)	Hardcoded secrets & insecure PRNG session tokens	Enforce secrets manager & cryptographically secure randoms
Tampering (T)	SQL injection & untrusted pickle deserialization	Parameterized queries & safe JSON/safe_load serializers
Repudiation (R)	Unauthenticated query execution & missing audit trail	Tamper-evident structured audit logging & trace headers
Info Disclosure (I)	Exposed private keys & path traversal directory leaks	Path canonicalization & environment variable isolation
Denial of Service (D)	Insecure object deserialization & dynamic eval hangs	Isolated AST evaluators & strict payload schema boundaries
Elevation of Priv (E)	OS command injection via shell=True & eval execution	Explicit argv list execution with shell=False & least privilege

5. Autonomous Patching & AST Safety Engine

How ForgeGuard eliminates the danger of automated refactoring breaking production systems:

■■ *Step 1: Intelligent Synthesis*	■ *Step 2: AST Validation Gate*
The Remediation Agent synthesizes secure replacements (parameterized SQL, safe subprocess lists, json.loads, environment variable lookups).	Before any modification is written to disk, the patched code is parsed through Python's `ast.parse()`. If syntax is invalid, the patch is instantly aborted.
■ *Step 3: Snapshot Backups*	■ *Step 4: 1-Click Rollback*
Atomic file snapshots are created in `.forgeguard_backups/` before any byte is modified on the filesystem.	Engineers can reverse any patch with a single CLI command (`forgeguard rollback`) or one click on the live dashboard.

6. Developer Experience & Enterprise Dashboard

Seamless interaction via modern CLI, REST API endpoints, and real-time Web Dashboard.

CLI Capabilities	*Web Dashboard*
• `forgeguard scan <dir>` (High-speed SAST) • `forgeguard audit <dir> --json report.json` • `forgeguard patch <dir> [--dry-run]` • `forgeguard serve <dir> --port 8899` • ASCII terminal banners & colored CVSS severity	• Real-time CVSS score & security grade gauges • Interactive vulnerability inspector & code snippets • Full STRIDE matrix grid with risk categorization • Side-by-side diff studio with 1-click Auto-Patch • Zero external web dependencies (pure standard lib)

7. Tech Stack, Performance & Roadmap

Core Tech Stack:	Python 3.11 Standard Library (`ast`, `http.server`, `difflib`, `socketserver`), Featherless AI Bridge (Llama-3.1-8B-Instruct), Tailwind CSS.
Scan Speed:	**< 5 ms** for standard projects; 100% deterministic SAST analysis with zero network dependencies required.
Testing:	100% test coverage with automated unit tests (`tests/test_forgeguard.py`) verifying all CWE classes and patch rollbacks.
Future Roadmap:	GitHub Actions CI/CD bot, multi-language AST support (TypeScript, Go, Rust), and automated pull request generation.